Project #3
STM32 Random Number Generator with EEPROM and LCD

(STM32 Microcontroller Practice)

1. Please implement the following functions for this project:
   (a) Generate a random number x.
   (b) Store the random number in the 24LC256 I²C EEPROM.
   (c) Read the random number back from the I²C EEPROM.
   (d) Display the random number on the LCD1602A in the format shown in Fig. 1.
   (e) Shift the random number across the display as a “news ticker.”

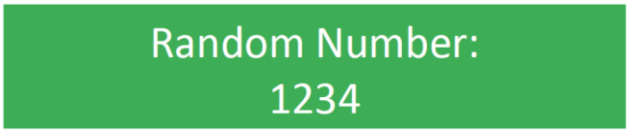

Figure 1. LCD display format.

© 2026 Hsu, Ting-Wei.

(i) Functional block diagram

- Nucleo-L476RG **產生 Random number**，將 Random number **寫入** 24LC256 I2C EEPROM。
- Nucleo-L476RG **讀取** 24LC256 I2C EEPROM 儲存的 Random number。
- Nucleo-L476RG 控制 LCD 1602A，並以 "News ticker" 的方式**顯示** Random number。
- 額外設置：每當 Push user button 時，就會**重新 Trigger** Nucleo-L476RG 執行產生亂數的整個流程。

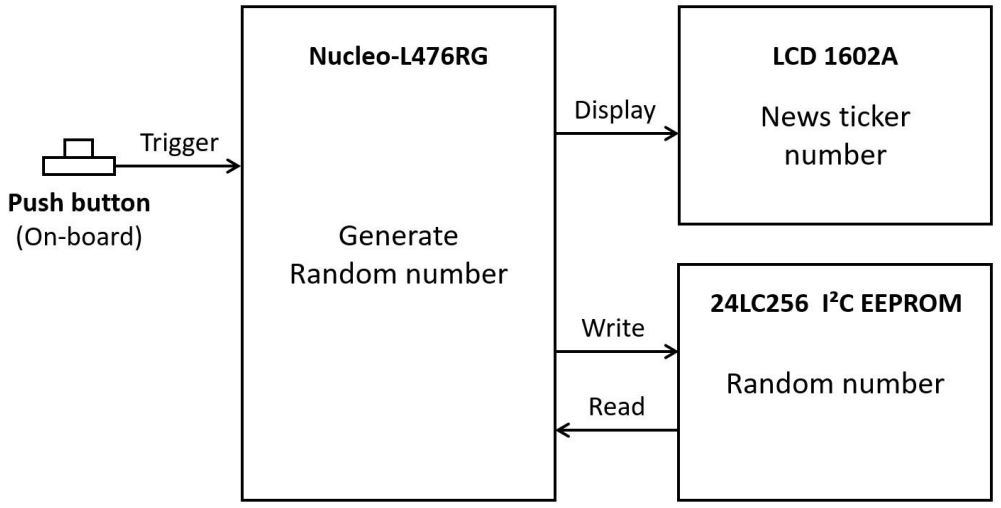

Fig. (1) The block diagram

© 2026 Hsu, Ting-Wei.

(ii) Design in a virtual breadboard using Tinkercad

- 繪製簡略 Circuit wiring diagram：

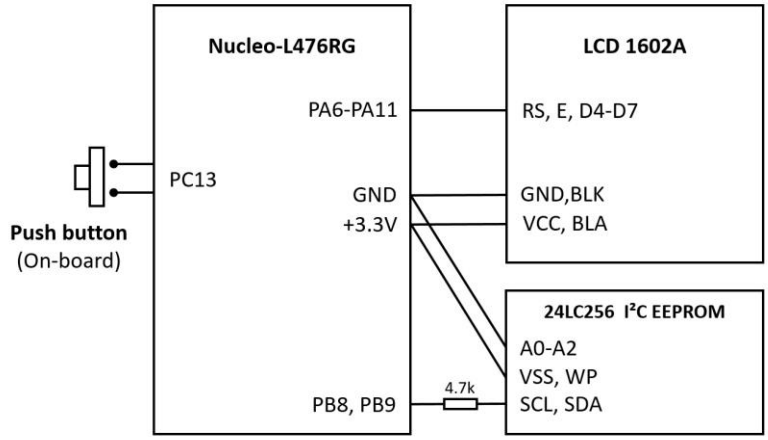

Fig. (2) The circuit wiring diagram

- 在 Tinkercad 實作 Circuit wiring：

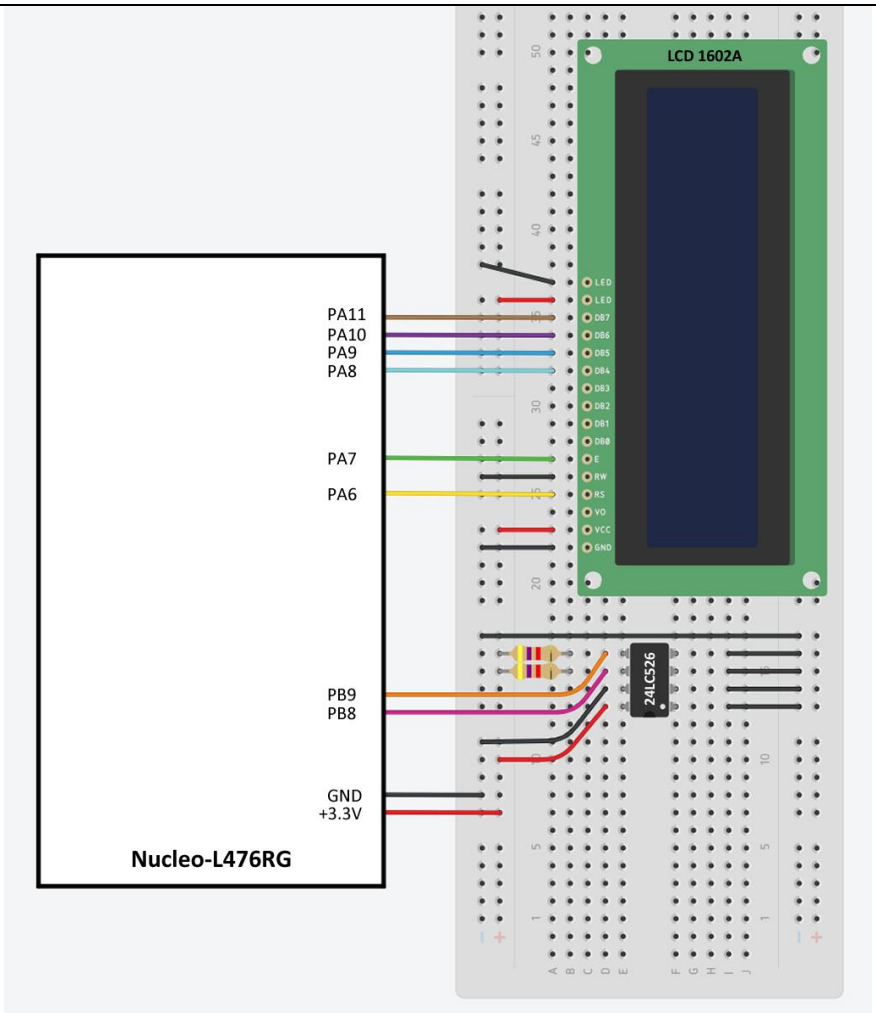

Fig. (3) The Tinkercad circuit

© 2026 Hsu, Ting-Wei.

(iii) Design in a real breadboard

- 實體 breadboard 實作 Circuit wiring photo 1：

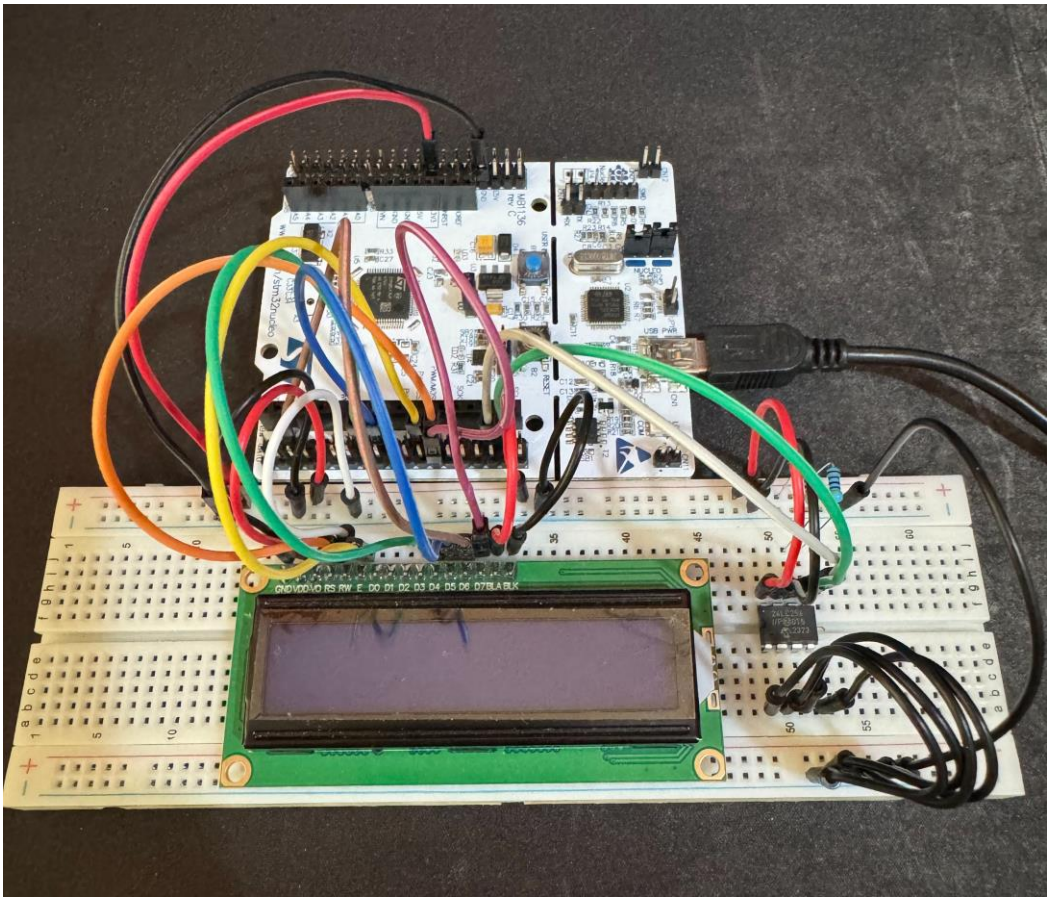

Fig. (4) The breadboard setup-1

© 2026 Hsu, Ting-Wei.

- 實體 breadboard 實作 Circuit wiring photo 2：

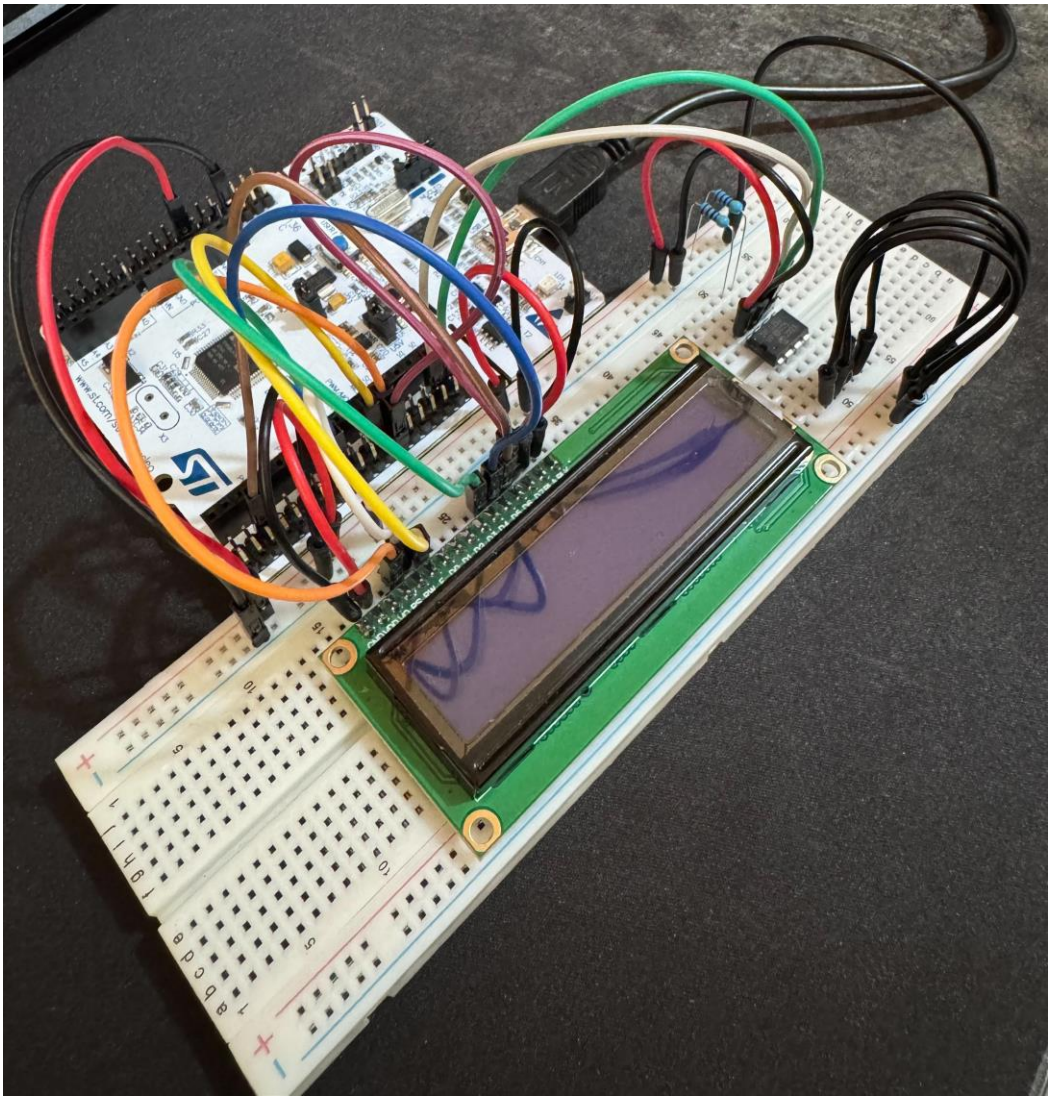

Fig. (5) The breadboard setup-2

© 2026 Hsu, Ting-Wei.

(iv) Codes and results

- **Results**
  - **Images]/demo.gif**
  - Video: https://youtube.com/shorts/FCf5yvo7H1Q?feature=share

(a) 上電後產生一組 4 位數的 Random number，在 LCD 上左右來回跑馬燈顯示。

(b) 按下 User button 後，重新產生新的一組 4 位數 Random number 在 LCD 上跑馬燈顯示。

Fig. (6) The LCD display result

© 2026 Hsu, Ting-Wei.

- **Codes**

1. 將 project 命名為 **RandomNumberStorageDisplay** 建置。

2. 設定 Nucleo-L476RG **IOC file** Pinout & Configuration：

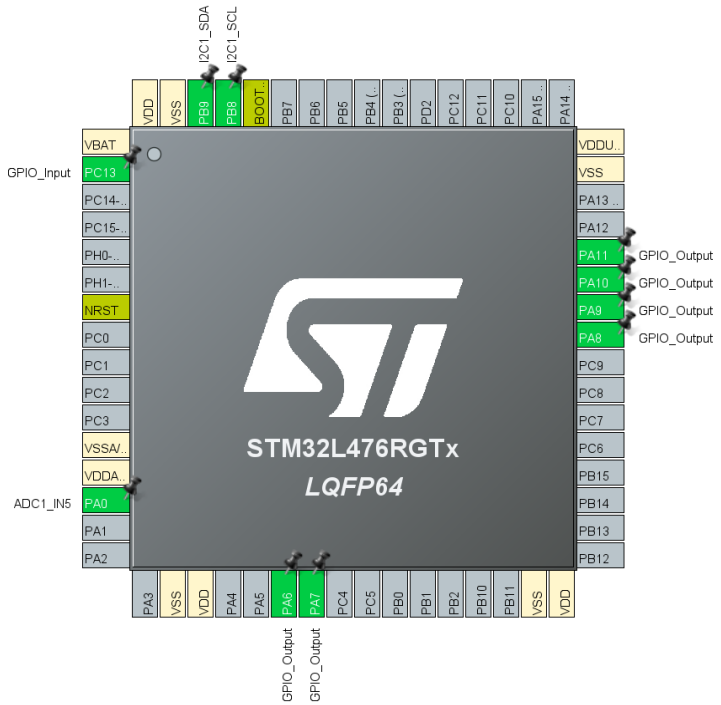

Fig. (7) The Pinout & Configuration settings

- PA6 – PA11 腳位用來連接 **LCD**，設置 GPIO_Output，以輸出顯示 Random number。
- PB8 – PB9 腳位設置 I2C1_SCL 和 I2C1_SDA，用來連接 **24LC256 EEPROM** 進行 I2C 傳輸通訊。
- 將 **User button** 的 PC13 腳位設置 GPIO_Input，用來取得 Button 按下狀態，重新產生 Random number。

- 在 Timer 裡的 **RTC** 中設置啟用 Activate Clock Source，用於計時基礎，作為**亂數產生種子**。

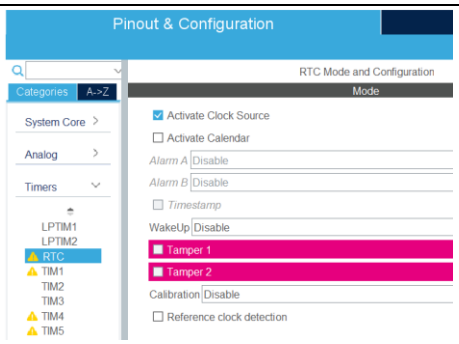

Fig. (8) The RTC random seed configuration

© 2026 Hsu, Ting-Wei.

- PA0 腳位設置 ADC1_IN5，在 **ADC1** 介面裡設置 IN5 Single-ended，用於取得腳位的類比雜訊，作為**亂數產生種子**。

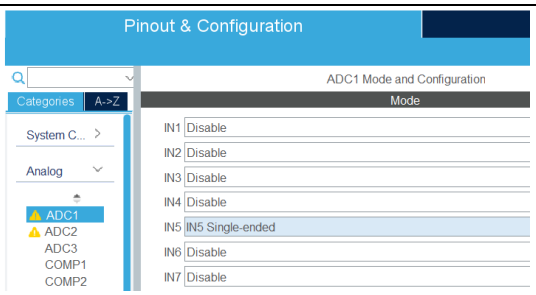

Fig. (9) The ADC random noise configuration

3. Codes

- **Generate Code** 後檢查生成設置正確，符合預期。

```
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9
                           |GPIO_PIN_10|GPIO_PIN_11, GPIO_PIN_RESET);

  /*Configure GPIO pin : PC13 */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pins : PA6 PA7 PA8 PA9
                          PA10 PA11 */
  GPIO_InitStruct.Pin = GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9
                           |GPIO_PIN_10|GPIO_PIN_11;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}
```

Fig. (10) The MX GPIO initialization

- 定義 `char ticker[5];` 用來存放要在 LCD 上顯示的 **Random number** (4 位數字元 + 1 位 '\0' 結束字元)。

- `Generate_Random_Number()`
  1. 取得 RTC 時間：
     `Seconds + Minutes + Hours + Date + Month + Year`
  2. 取得 ADC 類比雜訊值：
     `adc_value`
  3. 混合 (XOR) RTC 時間 ADC 類比雜訊值，作為亂數種子：
     `srand(seed)`
  4. 產生一組 0~9999 的 Random number：
     `random_number = rand() % 10000`

© 2026 Hsu, Ting-Wei.

- **EEPROM Write/Read**
  - `Write()` 將 `random_number` 寫入指定的記憶體位置

    ```
    Write(0x1000, (uint8_t *)&random_number, sizeof(random_number))
    ```
  - 設置 `HAL_Delay(10)` 緩衝，等待 EEPROM 寫入完成
  - `Read()` 從 EEPROM 讀取 `random_number`

    ```
    Read(0x1000, (uint8_t *)&random_number, sizeof(random_number))
    ```

- **跑馬燈**顯示 Random number
  - `int position = 6;` 用來記錄 ticker 在 LCD 的位置，初始值設為 LCD 的第 7 格，讓 Random number 初始顯示在中間。
  - `int move = -1;` 用來設置 ticker 在 LCD 的移動：-1 為往左移 1 格，+1 為往右移 1 格。
  - 在主程式 `while (1)`
    1. 設置 `HAL_Delay(200)`，每 200 毫秒刷新一次 Random number 顯示位置。
    2. 讓 Random number 在 LCD 移動：`position += move`
    3. `if(position <= 0)` 判斷當移動到**最左**邊 (第 0 格) 時，設置 `position = 0`，`move = 1`，改往右移動。
    4. `if(position >= 12)` 判斷當移動到**最右**邊 (第 12 格) 時，設置 `position = 12`，`move = -1`，改往左移動。
- User Button 按下後可以重新產生新 Random number 在 LCD 上跑馬燈顯示。為了防止 Button 按下後多次觸發，做了相關定義處理：
  - `button_state` 取得目前按下狀態 (0: 按下，1: 放開)。
  - `last_button_state` 記錄上次按鈕狀態，判斷是否剛被按下(是否由 1: 放開 → 變成 0: 按下)。
  - 為防止 Button 彈跳問題造成誤動作，按下後 `HAL_Delay(50)` 為 Debounce delay。

© 2026 Hsu, Ting-Wei.